K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

ALGORITHM STRATEGIES MINI PROJECT

Lab Assignment Number - 2

Submitted By:

Name	Mritunjay Tripathi
Roll Number	2401420017
Programme	B.Tech CSE (Data Science)
Department	Computer Science & Engineering
Session	2025–2026
Submitted To	Dr Aarti Sangwan
Course Name	Analysis and Design of Algorithm
Course Code	ENCS 256
School	School of Engineering & Technology
GitHub Link	https://github.com/mritunjaytripathi298-hash/algo-startegies-mini-project-MritunjayTripathi2401420017.git

KR Mangalam University

The Complete World of Education

1.Introduction

In this experiment, we explore different algorithmic paradigms including Divide and Conquer, Greedy Algorithms, and Dynamic Programming. These techniques are widely used in solving real-world optimization problems such as sorting, scheduling, and routing.
We analyse their performance based on execution time and scalability.

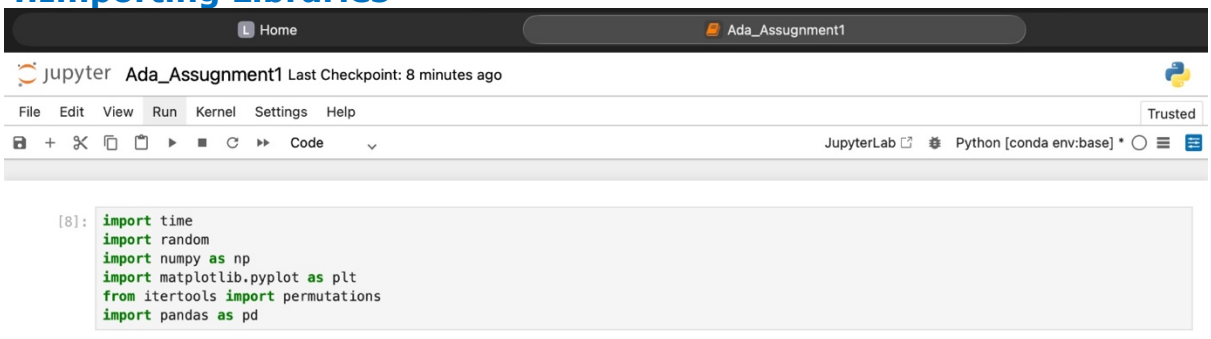
2.Objective

The objective of this experiment is to understand and implement different algorithmic paradigms such as Divide and Conquer, Greedy Algorithms, and Dynamic Programming. The experiment aims to analyse the performance of these algorithms by measuring execution time and observing scalability. Additionally, the experiment helps in understanding how different strategies are applied to solve real-world optimization problems efficiently.

3.Procedure

1. Set up the programming environment using Python and required libraries such as NumPy, Matplotlib, and Pandas.
2. Implement algorithms based on different paradigms:
   - Divide and Conquer (Merge Sort)
   - Greedy Algorithm (Activity Selection)
   - Dynamic Programming (Knapsack Problem)
   - Travelling Salesman Problem (TSP)
3. Generate random input data of different sizes to analyse performance.
4. Measure execution time using Python's time module.
5. Plot graphs to visualize performance comparison.
6. Analyse outputs and observe differences in time complexity and scalability.
7. Document results and include screenshots of outputs and graphs.

4.Importing Libraries



5.Divide and Conquer: Merge Sort

Objective

The objective of this experiment is to implement the Merge Sort algorithm using the Divide and Conquer technique and analyze its performance. The goal is to understand how recursive division and merging lead to efficient sorting with predictable time complexity.

Procedure

1. Generate an unsorted array of integers.
2. Divide the array into two halves recursively until each sub-array contains only one element.
3. Merge the divided sub-arrays in a sorted manner.
4. Continue merging until the entire array is sorted.
5. Measure the execution time for different input sizes.
6. Plot a graph to analyze performance.

Explanation

Merge Sort follows a Divide and Conquer approach:

- **Divide:** The array is recursively divided into two halves.
- **Conquer:** Each half is sorted individually.
- **Combine:** The sorted halves are merged into a single sorted array.

This recursive breakdown ensures that even large datasets are handled efficiently. The merging step compares elements from both halves and arranges them in order.

Merge Sort guarantees a time complexity of **O(n log n)** in all cases (best, average, worst), making it reliable for large datasets.

Performance Analysis

In this section, Merge Sort is compared with Python's built-in sorting function (sorted()), which uses the Time sort algorithm.

To conduct the comparison:

- Random arrays of varying sizes are generated.
- Both algorithms are applied to identical datasets.
- Execution time is recorded using precise timing functions.
- Results are plotted on a graph for better visualization.

Observations

- Python's built-in sort performs faster in practice due to:
  - Internal optimizations
  - Implementation in low-level language (C)
  - Adaptive nature (handles partially sorted data efficiently)
- Merge Sort shows consistent and predictable performance.
- As input size increases, Merge Sort maintains steady growth in execution time, reflecting its theoretical complexity

Code as follows:

```python
def merge(left, right):
    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged


def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


# Test
arr = [random.randint(1, 1000) for _ in range(100)]
start = time.perf_counter()
sorted_arr = merge_sort(arr)
end = time.perf_counter()

print("Sorted (first 10):", sorted_arr[:10])
print("Time Taken:", round(end - start, 6), "seconds")
```

Output as follows:

```
Sorted (first 10): [52, 83, 84, 98, 105, 121, 131, 141, 142, 145]
Time Taken: 0.000489 seconds
```

6.Sorting Performance Comparison

In this section, a detailed comparison is carried out between the Merge Sort algorithm (implemented using the Divide and Conquer paradigm) and Python's built-in sorting function (sorted()), which is based on the highly optimized Time sort algorithm.

The purpose of this comparison is to evaluate how different sorting approaches behave in terms of execution time when the size of the input data increases. This helps in understanding not only theoretical time complexity but also practical performance in real-world scenarios.

Methodology

To ensure a fair and consistent comparison, the following steps are performed:

- Random arrays of varying sizes (such as 100, 500, 1000, 2000, and 4000 elements) are generated using random number functions.
- For each input size, both algorithms are executed separately on identical datasets.
- The execution time is measured using a high-precision timer (time.perf_counter()), which provides accurate timing results.

- The measured times are stored and later visualized using graphs.

Visualization

The collected execution times are plotted on a graph with:

- **Input Size** on the x-axis
- **Execution Time** on the y-axis

This visual representation helps in clearly observing how each algorithm scales as the input size increases.

Explanation

- Merge Sort divides the array recursively and then merges sorted subarrays, ensuring a consistent time complexity of O(n log n).
- Python's built-in sorting function uses Timsort, which is a hybrid algorithm derived from Merge Sort and Insertion Sort, optimized for real-world data patterns.

Observations

From the graph and execution results, it can be observed that:

- Python's built-in sorting function consistently performs faster than the custom Merge Sort implementation.
- This improved performance is due to internal optimizations, such as:
    - Efficient handling of partially sorted data
    - Reduced overhead through low-level implementation in C
    - Adaptive behavior based on input patterns
- Merge Sort, although slightly slower in practice, shows stable and predictable performance across all input sizes.
- The execution time of Merge Sort increases in a smooth and consistent manner, reflecting its theoretical time complexity.

Conclusion of Comparison

This comparison highlights the importance of choosing the right implementation. While Merge Sort is theoretically efficient and reliable,

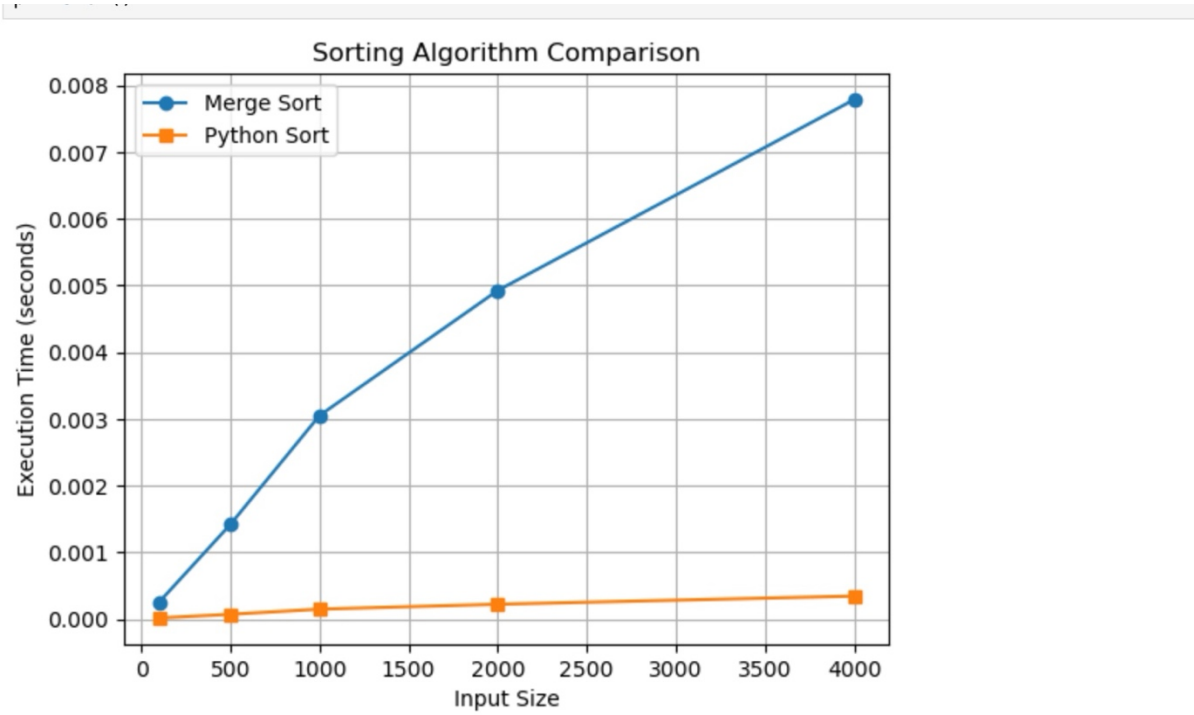
built-in functions often outperform custom implementations due to advanced optimizations and lower-level execution efficiency.

Thus, in practical applications, built-in sorting functions are generally preferred unless specific algorithmic control is required.

Code as follows:

```
[10]: sizes = [100, 500, 1000, 2000, 4000]

      merge_times = []
      builtin_times = []

      for size in sizes:
          data = [random.randint(1, 10000) for _ in range(size)]

          # Merge Sort timing
          start = time.perf_counter()
          merge_sort(data.copy())
          merge_times.append(time.perf_counter() - start)

          # Built-in sort timing
          start = time.perf_counter()
          sorted(data)
          builtin_times.append(time.perf_counter() - start)


      plt.figure()
      plt.plot(sizes, merge_times, marker='o', label="Merge Sort")
      plt.plot(sizes, builtin_times, marker='s', label="Python Sort")
      plt.xlabel("Input Size")
      plt.ylabel("Execution Time (seconds)")
      plt.title("Sorting Algorithm Comparison")
      plt.legend()
      plt.grid()
      plt.show()
```

Output as follows:



7.Activity Selection Problem (Greedy Algorithm)

Objective

The objective of this experiment is to implement the Activity Selection Problem using a Greedy approach and determine the maximum number of non-overlapping activities that can be performed.

Procedure

1. Define a list of activities with start and finish times.
2. Sort all activities based on their finishing times.
3. Select the first activity (earliest finishing time).
4. Iterate through the remaining activities:
    - Select an activity if its start time is greater than or equal to the finish time of the previously selected activity.
5. Store and display the selected activities.

Explanation

The Activity Selection Problem is solved using a Greedy strategy:

- At each step, the algorithm selects the activity that finishes earliest.
- This ensures that maximum time remains available for other activities.

The key idea is that making a locally optimal choice (choosing the earliest finishing activity) leads to a globally optimal solution.

The time complexity of this approach is **O(n log n)** due to sorting.

Observations (Elaborated)

- The greedy approach efficiently selects the maximum number of activities.
- Sorting based on finish time is crucial for correctness.
- The algorithm avoids unnecessary comparisons and ensures optimal scheduling.
- It performs well even for large input sizes.

Code as follows :

```
[11]: def activity_selection(start, finish):
          activities = list(zip(start, finish))

          # Sort by finish time
          activities.sort(key=lambda x: x[1])

          selected = [activities[0]]

          for i in range(1, len(activities)):
              if activities[i][0] >= selected[-1][1]:
                  selected.append(activities[i])

          return selected


      start = [1, 3, 0, 5, 8, 5]
      finish = [2, 4, 6, 7, 9, 9]

      result = activity_selection(start, finish)

      print("Selected Activities (start, end):")
      for act in result:
          print(act)
```

Output as follows :

```
Selected Activities (start, end):
(1, 2)
(3, 4)
(5, 7)
(8, 9)
```

8.Knapsack Problem (Dynamic Programming)

Objective

The objective of this experiment is to implement the 0/1 Knapsack Problem using Dynamic Programming to maximize the total value without exceeding the weight capacity.

Procedure

1. Define arrays for weights and values of items.
2. Set the maximum capacity of the knapsack.
3. Create a 2D DP table where rows represent items and columns represent capacities.
4. Fill the table using the recurrence relation:
    - If the current item's weight is less than or equal to capacity, choose the maximum of:
        - Including the item

        - Excluding the item
5. Extract the final result from the DP table.

Explanation

The 0/1 Knapsack Problem is solved using Dynamic Programming:

- Each item can either be included or excluded (no repetition allowed).
- The algorithm builds a table to store intermediate results, avoiding redundant calculations.

The recurrence relation is:

Value[i][w] = max(Value[i-1][w], Value[i-1][w - weight[i]] + value[i])

Time complexity: **O(n × W)**

Space complexity: **O(n × W)**

Observations

- Dynamic Programming ensures optimal selection of items.
- The algorithm avoids recomputation by storing previous results.
- It guarantees maximum value within the given capacity.
- Performance depends on both number of items and knapsack capacity.

Code as follows :

```python
def knapsack_dp(weights, values, capacity):
    n = len(values)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][capacity]


values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50

start = time.perf_counter()
result = knapsack_dp(weights, values, capacity)
end = time.perf_counter()

print("Maximum Value:", result)
print("Execution Time:", round(end - start, 6), "seconds")
```

Output as follows :

```
Maximum Value: 220
Execution Time: 0.000158 seconds
```

9.Travelling Salesman Problem (Brute Force)

Objective

The objective of this experiment is to solve the Travelling Salesman Problem (TSP) using the brute force approach and determine the minimum cost tour.

Procedure

1. Represent cities using a distance matrix.
2. Fix a starting city.
3. Generate all possible permutations of remaining cities.
4. Calculate the total distance for each possible route.
5. Select the route with the minimum total cost.

Explanation

The Travelling Salesman Problem aims to find the shortest path that visits all cities exactly once and returns to the starting point.

In the brute force approach:

- All possible permutations of cities are generated.
- Each route is evaluated.
- The minimum cost route is selected.

Time complexity: **O(n!)** (factorial growth)

This makes brute force impractical for large datasets but useful for understanding the problem.

Observations

The brute force method guarantees the optimal solution.

- Computation time increases rapidly as the number of cities increases.

- Suitable only for small datasets.
- Highlights the need for optimized approaches like Dynamic Programming or Heuristics for larger problems.

Code as follows :

```
def knapsack_dp(weights, values, capacity):
    n = len(values)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][capacity]


values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50

start = time.perf_counter()
result = knapsack_dp(weights, values, capacity)
end = time.perf_counter()

print("Maximum Value:", result)
print("Execution Time:", round(end - start, 6), "seconds")
```

Output as follows :

```
Maximum Value: 220
Execution Time: 0.000158 seconds
```

10.Exponential Growth

Objective

The objective of this section is to understand the concept of exponential growth and its significance in algorithm analysis, particularly in problems

like the Travelling Salesman Problem (TSP) and other brute-force approaches.

Explanation

Exponential growth refers to a situation where the rate of increase of a quantity is proportional to its current value. In simpler terms, the value grows rapidly as the input size increases, often doubling or increasing by a constant factor at each step.

In the context of algorithms, exponential growth occurs when the number of operations increases as a power of the input size. This is typically represented as:

$$T(n) = O(2^n), \quad O(3^n), \quad O(n!)$$

For example:

- If an algorithm takes 2^n steps, increasing input from 10 to 20 does not double the time — it increases it exponentially.
- In factorial growth $O(n!)$, the increase is even faster than exponential growth.

Example in Algorithms

- **Travelling Salesman Problem (Brute Force):**

  Requires checking all possible routes, leading to $O(n!)$ complexity.

- **Subset Generation Problems:**

  Often involve 2^n possibilities (each element can either be included or excluded).

Key Characteristics

- Growth rate is extremely fast.
- Small increases in input size result in very large increases in computation time.

- Becomes impractical for large inputs.

Observations

- Algorithms with exponential time complexity are inefficient for large datasets.
- They may work well for small input sizes but quickly become unusable as input grows.
- This type of growth highlights the importance of optimization techniques such as:
    - Dynamic Programming
    - Greedy Algorithms
    - Approximation Algorithms

Practical Understanding

If an algorithm takes:

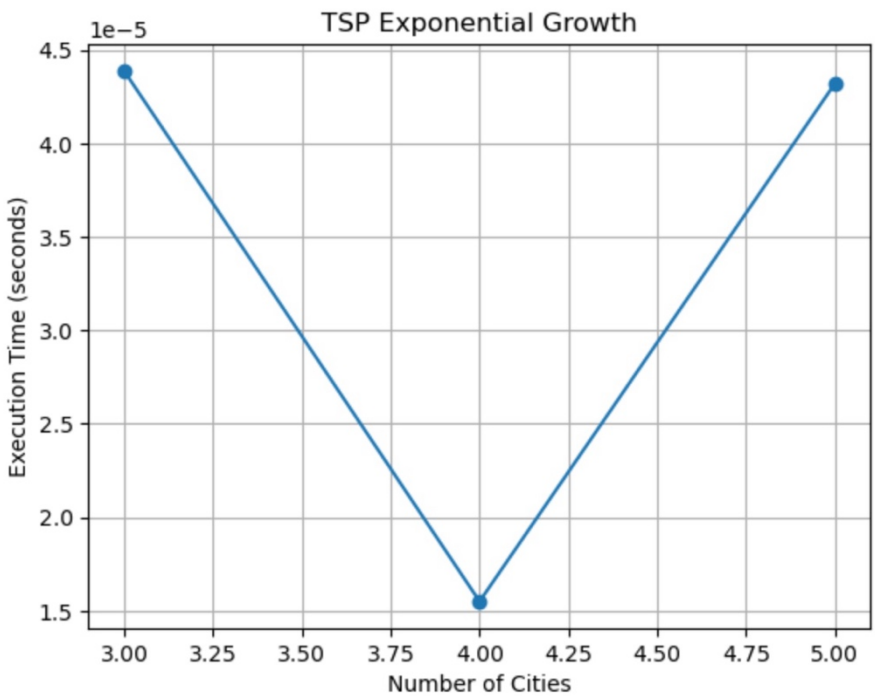
- 1 second for n = 10
- It may take hours or even years for n = 30 or more (depending on the growth rate)

Thus, exponential growth is considered one of the least efficient forms of algorithmic complexity.

Code as follows:

```python
def tsp_bruteforce(graph):
    n = len(graph)
    nodes = list(range(n))

    min_cost = float('inf')
    best_path = None

    for perm in permutations(nodes[1:]):
        current_cost = 0
        k = 0

        for j in perm:
            current_cost += graph[k][j]
            k = j

        current_cost += graph[k][0]

        if current_cost < min_cost:
            min_cost = current_cost
            best_path = (0,) + perm + (0,)

    return min_cost, best_path


graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

start = time.perf_counter()
cost, path = tsp_bruteforce(graph)
end = time.perf_counter()

print("Minimum Cost:", cost)
print("Best Path:", path)
print("Execution Time:", round(end - start, 6), "seconds")
```

Output as follows :



11.Algorithm Comparison

Code as follows :

```
data = {
    "Algorithm": ["Merge Sort", "Activity Selection", "Knapsack", "TSP"],
    "Technique": ["Divide & Conquer", "Greedy", "Dynamic Programming", "Brute Force"],
    "Complexity": ["O(n log n)", "O(n log n)", "O(nW)", "O(n!)"],
    "Use Case": ["Sorting", "Scheduling", "Optimization", "Routing"]
}

df = pd.DataFrame(data)
print(df)
```

Output as follows :

```
            Algorithm           Technique  Complexity      Use Case
0          Merge Sort     Divide & Conquer  O(n log n)       Sorting
1  Activity Selection               Greedy  O(n log n)    Scheduling
2            Knapsack  Dynamic Programming       O(nW)  Optimization
3                 TSP          Brute Force       O(n!)       Routing
```

Algorithm	Technique	Time Complexity	Use Case
Merge Sort	Divide and Conquer	O(n log n)	Sorting
Activity Selection	Greedy	O(n log n)	Scheduling
Knapsack	Dynamic Programming	O(nW)	Optimization
TSP	Brute Force	O(n!)	Routing

Observation:

Each algorithm is suitable for specific types of problems. Choosing the correct algorithm improves efficiency significantly.

12.Reflection & Discussion

Trade-off Analysis

Divide and Conquer provides efficient solutions for large datasets.

Greedy algorithms are fast but may not always produce optimal solutions.

Dynamic Programming ensures optimal solutions but uses more memory.

Real-world Suitability

Merge Sort is widely used in data processing.

Greedy algorithms are used in scheduling and network routing.

Dynamic Programming is applied in optimization problems.

TSP is useful in route planning but limited due to high complexity.

Recursion Depth

Divide and Conquer algorithms rely on recursion, which may lead to stack overflow if input size becomes very large.

Conclusion

This experiment demonstrated the implementation and comparison of different algorithmic strategies. Each technique has its advantages and limitations depending on the problem type.

Divide and Conquer is efficient for sorting, Greedy algorithms are fast for specific optimization problems, and Dynamic Programming guarantees optimal solutions. However, problems like TSP highlight the limitations of brute-force approaches due to exponential growth.

Thus, selecting the appropriate algorithm is essential for achieving efficient and scalable solutions.